

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..30

CUSTOM PROVIDERS

 Copy as Markdown

Custom providers

A custom identity provider plugs your own sign-in method into the authentication module. It issues the same tokens, creates the same auth users, and appears to your app like any built-in provider. Build one when the provider you need is not among the built-in ones, or when you authenticate against your own user directory.

A custom provider consists of a few parts:

- A **provider class** that implements the `IdentityProvider` contract on the server.
- A **config class** extending `IdentityProviderBuilder`, which you pass to `initializeAuthServices` like any built-in config.
- An **endpoint** extending `IdpBaseEndpoint`, which your app calls to sign in.
- An **account model** linking the provider's user identity to the Serverpod auth user.
- A **controller** in your app that runs the sign-in flow and registers the returned session.

How to build them depends on the kind of provider:

- **OAuth2-based providers** (most third-party services): the module ships utilities that handle the PKCE flow, token exchange, and error handling on both sides. Start with the [OAuth2 utility setup](#), then follow [creating an OAuth2-based identity provider](#) for the full walkthrough.
- **Everything else** (your own credential store, an internal single sign-on system, an API-key exchange): the same parts apply, without the OAuth2 utilities. The walkthrough still shows the shape of each part.

To replace the authentication module entirely rather than add a provider to it, see [custom overrides](#) instead.

Related

- [OAuth2 utility setup](#): the client and server utilities for OAuth2 flows.
- [Creating an OAuth2-based identity provider](#): the complete walkthrough.
- [Custom overrides](#): replace the built-in authentication entirely.



